

Report on
CHARISMART
Donation & Surplus Food Redistribution Network
CSE-3208: System Analysis & Design Lab

Submitted To
Md Mynoddin
Assistant Professor
Dept. of CSE, RMSTU

Submitted By

Team	Member / ID	Responsibility
Null Pointer	Ovi Das - 2101011035	Project Manager & Developer
	Akhaiye Marma - 2101011012	System Analyst & Developer
	Annah Tripura - 2101011015	UI/UX Designer
	Abu Arif Naim - 2101011034	QA / Tester



Department of Computer Science and Engineering
Rangamati Science and Technology University

Table of Contents

1. Executive Summary	5
2. Problem Statement & Motivation	6
2.1 Background	6
2.2 Identified Problems	6
2.3 How ChariSmart Solves These Problems	6
2.4 Current (As-Is) Process — Before ChariSmart.....	7
3. Stakeholder Analysis.....	8
3.1 Stakeholder Identification.....	8
3.2 Role Capabilities Summary	8
4. Feasibility Study	9
4.1 Technical Feasibility	9
4.2 Economic Feasibility.....	9
4.3 Operational Feasibility	9
4.4 Schedule Feasibility	9
4.5 Legal / Ethical Feasibility.....	9
4.6 Go / No-Go Decision	10
5. System Overview & Architecture.....	11
5.1 Technology Stack	11
5.2 Application Entry Point & Configuration	11
5.3 Request Lifecycle	11
5.4 URL Route Map	12
6. Database Design	13
6.1 Models Overview	13
6.2 Relationships	13
6.3 Area Normalisation.....	14
6.4 Normalisation Justification (1NF–3NF).....	14
6.5 Backup & Recovery Strategy.....	15
7. Software Requirements Specification (SRS)	16
7.1 Functional Requirements	16
7.2 Non-Functional Requirements	16
7.3 Requirements Prioritisation (MoSCoW)	17
7.4 Requirements Traceability Matrix (RTM)	17
8. Data Flow Diagrams (DFD).....	18
8.1 Level 0 — Context Diagram.....	18

8.2 Level 1 — System Decomposition	18
8.3 Level 2 — Approval & Notification Engine (P3 Detail)	19
9. Entity-Relationship Diagram (ERD).....	20
9.1 Entities, Attributes, and Keys	20
10. UML Diagrams	21
10.1 Use Case Diagram — Summary.....	21
10.2 Detailed Use Case Descriptions	22
Use Case: Submit Surplus Food Donation	22
Use Case: Verify & Approve / Reject Donation	22
Use Case: Auto-Match Donation to Request	22
10.3 Activity Diagram — Complete Donation Lifecycle.....	23
10.4 Sequence Diagram — Admin Approves Donation	24
10.5 Class Diagram — Key Classes.....	25
10.6 State Diagrams	26
10.7 Package Diagram	27
11. Module & Feature Description	28
11.1 Authentication Module	28
11.2 Donation Module	28
11.3 Notification & Email Module	28
11.4 Auto-Expiry Module	28
11.5 Food Request Module	28
12. UI/UX Design Description.....	29
12.1 Design Overview	29
12.2 Template Structure.....	29
12.3 Pages and Key UI Elements.....	29
12.4 Status Badge Colours	29
12.5 User Personas	30
12.6 Simplified Navigation Flow	30
12.7 Accessibility Notes	30
13. Implementation Details.....	31
13.1 Running the Application.....	31
13.2 Environment Variables	31
13.3 Key Implementation Patterns	31
13.4 Coding Standards.....	32
13.5 Folder Structure	32
14. Testing Strategy	33

14.1 Testing Approach.....	33
14.2 Test Cases	33
14.3 Additional Testing Notes	33
15. Risk Analysis & Mitigation	34
16. References	35

1. Executive Summary

ChariSmart is a web-based surplus-food redistribution platform built to reduce food waste and support vulnerable communities in urban areas. The system connects three donor categories — restaurants, wedding halls and households — with NGOs/orphanages that submit structured food requests, and coordinates the last-mile handover through a volunteer/rider role.

The prototype is implemented in Flask with a SQLite database and server-rendered Jinja2 templates. It provides role-based dashboards for Donor, Recipient, Volunteer and Admin; an admin-controlled verification and approval workflow for both donations and requests; an automatic area-based matching engine that scores candidate donation–request pairs; volunteer assignment and pickup/delivery tracking; an expiry-aware scheduler that auto-expires stale donations; and an email/SMS notification layer (Gmail SMTP and optional Twilio) that degrades gracefully to a terminal-printed notification when no credentials are configured.

This report documents the complete System Analysis & Design (SAD) lifecycle of the project: problem definition and motivation, stakeholder analysis, feasibility study, system architecture, database design with normalisation justification, the Software Requirements Specification (functional and non-functional requirements, MoSCoW prioritisation and a requirements traceability matrix), Data Flow Diagrams at three levels of decomposition, an Entity-Relationship Diagram, a full set of UML diagrams (use case, activity, sequence, class, state and package), module-by-module feature description, UI/UX design description, implementation details, a testing strategy with executed test cases, and a risk register with mitigation strategies.

The core value proposition of ChariSmart, relative to ad-hoc coordination through phone calls or social media posts, is traceability and safety: every donation and request passes through admin verification before it is matched, every match is scored on area, quantity and priority, every handover is tracked through explicit status transitions (Pending → Approved → Matched → Assigned → Picked Up → Delivered, or Rejected/Expired), and every status change generates a notification so no stakeholder is left waiting without information.

2. Problem Statement & Motivation

2.1 Background

In many urban areas, edible surplus food from restaurants, wedding halls and households goes to waste while NGOs, orphanages and vulnerable families face food shortages on the same day, often within the same neighbourhood. Existing coordination between these two sides is largely informal — phone calls, WhatsApp/Facebook posts, or personal contacts — which does not scale, is not auditable, and provides no guarantee that food will be collected before it becomes unsafe to eat.

2.2 Identified Problems

- No central place to see what surplus food is available right now, in which area, and until when it remains safe to collect.
- No verification step: anyone can claim to be a genuine donor or a genuine NGO, which creates trust and food-safety risk.
- Manual matching is slow and area-blind — a donor in one part of the city may be matched (informally) with a recipient far away, increasing the chance the pickup window is missed.
- No structured tracking of the handover: it is unclear whether a donation has actually been picked up, delivered, or has silently expired.
- No notification trail — donors and recipients are often left waiting without knowing whether their submission was accepted, rejected, or matched.
- Duplicate or conflicting requests from multiple NGOs for the same donation are not detected.

2.3 How ChariSmart Solves These Problems

- A single web platform where donors submit food with quantity, expiry time and pickup window, and recipients submit requests with beneficiary group, quantity needed and a needed-before deadline.
- Admin-controlled verification: users can be marked verified/blocked, and every donation/request must be explicitly approved or rejected by an admin before it can be matched.
- An automatic, area-aware matching engine (`calculate_match_score`) that ranks candidate requests for an approved donation by area match, quantity fit and request priority.
- An explicit status lifecycle for every donation and match (Pending → Approved → Matched → Assigned → Picked Up → Delivered / Rejected / Expired) so every stakeholder can see exactly where a handover stands.
- A notification layer that fires on every decision and status change — in-app notification row, email via SMTP, and optional SMS via Twilio — with a terminal-print fallback so the workflow is demonstrable without live credentials.
- A background auto-expiry check (`auto_expire_donations`) that marks a donation Expired once its expiry time passes without delivery, preventing unsafe food from silently lingering in the pipeline.

2.4 Current (As-Is) Process — Before ChariSmart

Before a system such as ChariSmart, the typical coordination process observed in the problem domain follows an unstructured, manual pattern:

1. A restaurant, wedding hall or household has surplus food and calls or messages a personal contact at an NGO, or posts on social media.
2. An NGO representative sees the message (if at all) and calls back to confirm availability, quantity and pickup address.
3. No independent party verifies that the food is safe or that the requesting NGO genuinely needs it — trust is based purely on personal relationships.
4. A volunteer or the donor's own staff is asked, on an ad-hoc basis, to transport the food; there is no dispatch record.
5. There is no confirmation step: the donor often does not know whether the food was actually delivered, and no rating or feedback is captured.
6. If the intended recipient cannot collect in time, the food is frequently wasted because there is no automatic fallback to another nearby NGO.

This as-is process is slow, non-auditable, area-blind and provides no safety net when a match falls through — which is precisely the gap ChariSmart's structured, admin-verified, notification-driven workflow is designed to close.

3. Stakeholder Analysis

3.1 Stakeholder Identification

Stakeholder	Interest / Responsibility
Donor (Restaurant / Wedding Hall / Household)	Posts surplus food with safety details, quantity, expiry time and pickup window; receives status notifications.
Recipient NGO / Orphanage	Requests food for beneficiaries with quantity needed and a needed-before deadline; confirms delivery and may rate the handover.
Volunteer / Rider	Accepts assignment, collects food from the donor and updates pickup and delivery status.
Admin	Verifies/blocks users, approves or rejects donations and requests, triggers auto-matching, assigns volunteers, and manages the overall workflow.
Beneficiaries	Indirect stakeholders who ultimately receive redistributed food through the recipient organisation.
Course Teacher / Evaluator	Evaluates the SAD documentation, the working prototype, and the project defense.

3.2 Role Capabilities Summary

Role	Can Do
Donor	Register/login; submit donation; view own donations and their status; delete a still-pending donation.
Recipient	Register/login; submit food request; view own requests and matches; confirm delivery and add a rating/note.
Volunteer	Register/login; view assigned matches; mark pickup confirmed; mark delivery confirmed.
Admin	Verify/block/unblock/delete users; approve/reject donations and requests (with optional reason); trigger auto-match; assign a volunteer to a match; view system-wide stats.

4. Feasibility Study

4.1 Technical Feasibility

The prototype is built entirely on lightweight, well-documented open-source technology: Python 3 with Flask as the web framework, Flask-SQLAlchemy as the ORM, SQLite as the embedded database, Jinja2 templates with hand-written HTML/CSS for the UI, and Python's smtplib plus an optional Twilio REST call for notifications. Every dependency is listed in requirements.txt and installs cleanly in a standard virtual environment, so the technical risk of the stack itself is low. The team already has working familiarity with Flask and relational modelling, which further supports technical feasibility.

4.2 Economic Feasibility

Development cost is limited to developer time; there is no licensing cost because every component (Flask, SQLAlchemy, SQLite, Jinja2) is open source. Running the prototype locally has no hosting cost, and even a small-scale production deployment (e.g. a single low-cost VPS with a managed PostgreSQL instance) would remain inexpensive relative to the manual coordination effort it replaces. The main recurring cost in a real deployment would be optional SMS credits via Twilio, which is opt-in and can be left disabled without breaking the core workflow.

4.3 Operational Feasibility

The workflow implemented in ChariSmart mirrors the real donation-coordination process the stakeholders already follow informally (submit → verify → match → assign → deliver), so no stakeholder needs to learn a fundamentally new way of working — only a structured, faster version of what they already do. Role-based dashboards keep each actor's view focused on the actions relevant to them, which supports adoption by non-technical NGO staff and volunteers.

4.4 Schedule Feasibility

The project was scoped to fit a single lab-course timeline by prioritising a minimum viable set of workflows first (registration, donation/request submission, admin approval, matching, delivery tracking) and layering notifications, auto-expiry and richer admin tooling on top once the core loop was functional. This staged approach kept the schedule realistic for a student team working alongside other coursework.

4.5 Legal / Ethical Feasibility

Because the system handles food destined for vulnerable beneficiaries, food-safety verification is treated as an admin responsibility rather than an automated guarantee — the platform does not claim to certify food safety, only to route verified submissions through a transparent, auditable process. User data (contact details, addresses) is only shown to the parties involved in a given match, passwords are stored as salted hashes (Werkzeug's `generate_password_hash/check_password_hash`), and notification credentials (SMTP/Twilio) are kept in a local `.env` file that is explicitly excluded from version control.

4.6 Go / No-Go Decision

Feasibility Dimension	Verdict
Technical	Go — lightweight, proven stack; team has working familiarity.
Economic	Go — zero licensing cost; optional SMS is the only recurring cost.
Operational	Go — mirrors the stakeholders' existing informal workflow.
Schedule	Go — MVP-first staging fits the lab timeline.
Legal / Ethical	Go, with the caveat that food-safety certification remains a human (admin) responsibility, not an automated one.

Overall decision: GO. The project is feasible as a System Analysis & Design lab deliverable and as a demonstrable working prototype.

5. System Overview & Architecture

5.1 Technology Stack

Layer	Technology	Purpose
Backend Framework	Flask (Python)	Routing, request handling, session management, template rendering.
ORM / Database	Flask-SQLAlchemy over SQLite (charismart.db)	Object-relational mapping; DATABASE_URL env var allows swapping to PostgreSQL/MySQL.
Templating	Jinja2	Server-rendered HTML for every dashboard and form.
Frontend	Hand-written HTML5 + static/style.css	Role-based dashboards, forms, status badges, responsive layout.
Auth	Werkzeug security (generate_password_hash / check_password_hash) + Flask session	Password hashing and login-session based authentication.
Email	Python smtplib + email.message.EmailMessage	Sends decision notifications via Gmail SMTP (or prints to terminal if unset).
SMS (optional)	Twilio REST API via urllib	Sends SMS notifications when TWILIO_* credentials and NOTIFICATION_SMS_ENABLED are set.
Configuration	python-dotenv (.env / .env.example)	Keeps secrets and SMTP/Twilio credentials out of source control.

5.2 Application Entry Point & Configuration

app.py is the single-file Flask application entry point. On start-up it loads environment variables with `load_dotenv()`, configures `SECRET_KEY` and `SQLALCHEMY_DATABASE_URI` (defaulting to `sqlite:///charismart.db`), disables `SQLALCHEMY_TRACK_MODIFICATIONS`, and initialises the SQLAlchemy db object bound to the app. Constant lists define the closed vocabulary for the system: `ROLES` (admin, donor, recipient, volunteer), `DONOR_TYPES` (restaurant, wedding_hall, household, other), and the status vocabularies `DONATION_STATUSES`, `REQUEST_STATUSES` and `MATCH_STATUSES`. A `before_every_request` hook runs `auto_expire_donations()` on every incoming request so expired donations are cleaned up continuously rather than through a separate scheduled job.

5.3 Request Lifecycle

7. A request arrives at Flask's WSGI entry point and is routed by URL rule to the matching view function.
8. `before_every_request()` fires first: it calls `auto_expire_donations()`, which marks any Approved/Matched/Assigned donation whose `expire_at` has passed as Expired.

9. If the route is decorated with `@login_required`, `current_user()` is resolved from the Flask session; an unauthenticated request is redirected to `/login`.
10. If the route is further decorated with `@role_required(role)`, the current user's role is checked and a 403-style redirect/flash occurs on mismatch (for example, only admin may call the approval routes).
11. The view function executes business logic against the SQLAlchemy models, commits the session, and — for state-changing admin actions — calls `notify_user()/decision_notification()` to create a Notification row and dispatch email/SMS.
12. A Jinja2 template is rendered (or a redirect issued) and the response is returned to the browser; `inject_user()` makes the current user available to every template for the navbar.

5.4 URL Route Map

Route	Methods	Purpose
<code>/</code>	GET	Public landing page (home).
<code>/register</code>	GET, POST	Create a new donor / recipient / volunteer account.
<code>/login</code>	GET, POST	Authenticate and start a session.
<code>/logout</code>	GET	Clear the session.
<code>/dashboard</code>	GET	Role-aware dashboard dispatch (donor / recipient / volunteer / admin views).
<code>/donate</code>	GET, POST	Donor submits a new surplus-food donation.
<code>/request-food</code>	GET, POST	Recipient submits a new food request.
<code>/requests</code>	GET	Browse/list food requests.
<code>/admin/request/<id>/<status></code>	POST	Admin approves or rejects a food request.
<code>/admin/donation/<id>/<status></code>	POST	Admin approves or rejects a donation.
<code>/admin/donation/<id>/auto-match</code>	POST	Admin triggers the auto-matching engine for a donation.
<code>/admin/match/<id>/assign</code>	POST	Admin assigns a volunteer to a FoodMatch.
<code>/match/<id>/<status></code>	POST	Volunteer/recipient updates a match's status (pickup, delivered).
<code>/donation/<id>/delete</code>	POST	Donor deletes their own still-pending donation.
<code>/admin/users</code>	GET	Admin: list and manage all users.
<code>/admin/user/<id>/toggle-block</code>	POST	Admin blocks/unblocks a user.
<code>/admin/user/<id>/toggle-verified</code>	POST	Admin verifies/un-verifies a user.
<code>/admin/user/<id>/delete</code>	POST	Admin deletes a user.

6. Database Design

6.1 Models Overview

The system persists five SQLAlchemy models: User, Donation, FoodRequest, FoodMatch and Notification. Each model maps to one table in the SQLite database (charismart.db by default; DATABASE_URL can point elsewhere).

Model	Key Columns	Notes
User	id (PK), name, email (unique), password_hash, role, area, phone, organization_type, capacity_meals, verified, is_blocked, created_at	role is one of admin/donor/recipient/volunteer; verified/is_blocked are admin-controlled flags.
Donation	id (PK), donor_id (FK→User), food_name, donor_type, quantity_meals, area, pickup_address, pickup_window_start/end, expire_at, status, created_at	status cycles through pending → approved → matched → assigned → picked_up → delivered, or rejected/expired.
FoodRequest	id (PK), recipient_id (FK→User), beneficiary_group, requested_food_type, quantity_needed, area, delivery_address, needed_before, priority, status, created_at	priority feeds the match-score calculation.
FoodMatch	id (PK), donation_id (FK→Donation), request_id (FK→FoodRequest), volunteer_id (FK→User, nullable), meals_allocated, match_score, status, pickup_confirmed_at, delivered_at, recipient_rating, created_at	The join entity linking one donation, one request and (once assigned) one volunteer.
Notification	id (PK), receiver_email, title, message, created_at	In-app notification record; also mirrored to email/SMS by the notification service.

6.2 Relationships

- User (1) — (M) Donation: a donor posts many donations (User.posts).
- User (1) — (M) FoodRequest: a recipient submits many requests (User.submits).
- Donation (1) — (M) FoodMatch: a donation can, in principle, be referenced by matches over its lifecycle (matched_by).
- FoodRequest (1) — (M) FoodMatch: a request can be referenced by matches (matched_to).
- User (1) — (M) FoodMatch as volunteer: a volunteer can be assigned to many matches (nullable until assignment).
- User (1) — (M) Notification: every notification is addressed to one user's e-mail (receiver_email).

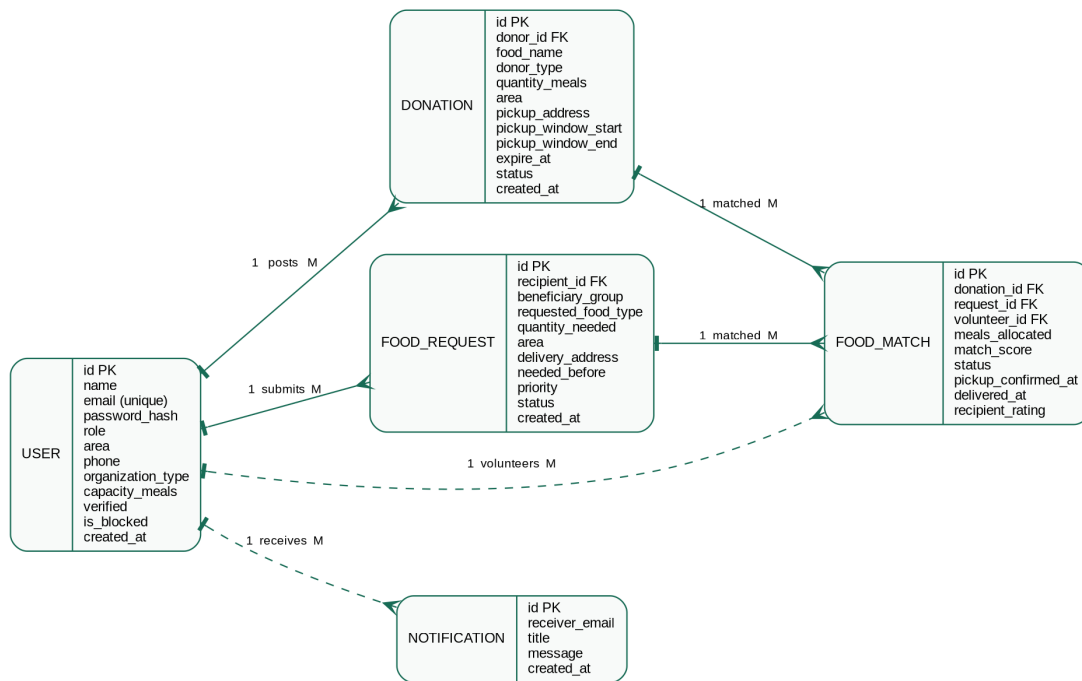


Figure 6.2 Entity-Relationship Diagram of the ChariSmart schema.

6.3 Area Normalisation

Donor pickup area and recipient delivery area are free-text fields (area column on both Donation and FoodRequest), but the matching engine relies on comparing them for equality. To make that comparison reliable, the application applies a `normalize_area()` helper that trims whitespace and lower-cases the value before it is stored and before it is compared during auto-matching, so 'Dhanmondi', ' dhanmondi ' and 'DHANMONDI' are treated as the same area without requiring a separate lookup table in this prototype.

6.4 Normalisation Justification (1NF–3NF)

- 1NF: Every column in every table holds a single, atomic value (e.g. `quantity_meals` is one integer, not a list); there are no repeating groups — a donor's multiple donations are separate rows in Donation rather than a multi-valued column on User.
- 2NF: All five tables use a single-column surrogate primary key (`id`), so there are no composite keys and therefore no possibility of a partial dependency on part of the key; every non-key column depends on the whole of `id`.
- 3NF: Non-key attributes depend only on the primary key and not on each other. For example, FoodMatch stores `match_score` and `status` against its own `id` rather than duplicating donor/recipient details, which are instead reached via `donation_id/request_id/volunteer_id` foreign keys — this avoids update anomalies (e.g. changing a donor's phone number in one place, on User, rather than in every Donation row).

6.5 Backup & Recovery Strategy

- Development: the SQLite file `charismart.db` can be copied as a single-file backup at any time; this is sufficient for a lab prototype.
- Production recommendation: point `DATABASE_URL` at a managed PostgreSQL/MySQL instance and rely on the provider's automated point-in-time backups, or run a scheduled `pg_dump/mysqldump`.
- Recovery: restoring the copied SQLite file (or a database dump) and restarting the Flask app is sufficient to recover the full application state, since all business data lives in the five relational tables described above.

7. Software Requirements Specification (SRS)

7.1 Functional Requirements

ID	Requirement
FR-01	Users can register as donor, recipient NGO/orphanage, or volunteer, and log in/out.
FR-02	Admin can verify, block/unblock and delete users.
FR-03	Donor can submit surplus food with donor type, quantity, area, pickup address, pickup window and expiry time.
FR-04	Recipient can submit a food request with beneficiary group, quantity needed, area, delivery address and needed-before time.
FR-05	Admin can approve or reject a donation or a request, optionally with a reason/note.
FR-06	System can auto-match an approved donation with the best-scoring pending request in the same area.
FR-07	Admin can assign an available volunteer to a created match.
FR-08	Volunteer can confirm pickup and confirm delivery for an assigned match.
FR-09	Recipient can confirm delivery and optionally submit a rating/note.
FR-10	System sends an in-app notification, an email, and (if configured) an SMS whenever a donation/request is approved or rejected.
FR-11	System automatically marks a donation Expired once its expiry time passes without being delivered.
FR-12	Donor can delete their own donation while it is still pending.

7.2 Non-Functional Requirements

Category	Requirement
Security	Passwords are stored as salted hashes; role-based access control restricts admin-only routes; secrets live in a git-ignored .env file.
Usability	Role-specific dashboards keep each actor focused on relevant actions; status badges use consistent colour coding across the UI.
Reliability	Notification sending failures (e.g. missing SMTP credentials) degrade gracefully to a terminal-printed message rather than crashing the request.
Maintainability	A single, well-commented app.py with clearly separated model, helper and route sections; DATABASE_URL and all credentials are externalised via environment variables.
Performance	SQLite and simple indexed lookups are sufficient for the expected prototype/demo load; the matching query scope is limited to pending items in one area.
Portability	The Flask app runs unmodified on Windows/macOS/Linux inside a standard virtual environment; swapping SQLite for PostgreSQL only requires changing DATABASE_URL.

7.3 Requirements Prioritisation (MoSCoW)

Priority	Requirements
Must Have	FR-01, FR-02, FR-03, FR-04, FR-05, FR-08, FR-09 — the core submit → verify → deliver loop.
Should Have	FR-06, FR-07, FR-10 — auto-matching, volunteer assignment and notifications, which greatly improve efficiency but a manual fallback is conceivable.
Could Have	FR-11, FR-12 — auto-expiry and self-service donation deletion, which improve data hygiene but are not required for a single demo transaction.
Won't Have (this release)	Payment/financial donation processing; real-time GPS navigation; AI-based route optimisation; government food-safety certification integration.

7.4 Requirements Traceability Matrix (RTM)

Req. ID	Source (Stakeholder Need)	Design Artifact	Test Case
FR-01	Stakeholder Analysis §3.1 — all roles need accounts	ERD: User; Use Case: Register/Login	TC-02, TC-03, TC-04
FR-02	Problem §2.2 — no verification of genuine users	Use Case: Manage Users; Module §11.6	TC-05
FR-03	Problem §2.2 — no central place to list surplus food	ERD: Donation; DFD P1.0	TC-06, TC-07
FR-04	Problem §2.2 — NGOs need a structured request channel	ERD: FoodRequest; DFD P2.0	TC-08
FR-05	Problem §2.2 — no verification before matching	DFD P3.0 (Level 2); Sequence Diagram	TC-09, TC-10
FR-06	Problem §2.2 — manual matching is area-blind and slow	Class Diagram: FoodMatch.calculate_score()	TC-11
FR-07	Problem §2.2 — no structured dispatch record	Use Case: Assign Volunteer; Activity Diagram	TC-12
FR-08	Problem §2.2 — pickup/delivery not tracked	State Diagram: Assigned→Picked Up→Delivered	TC-13, TC-14
FR-09	Stakeholder need — recipients confirm & rate	ERD: FoodMatch.recipient_rating	TC-14
FR-10	Problem §2.2 — stakeholders left waiting without info	Module §11.3 Notification & Email Module	Manual checklist
FR-11	Problem §2.2 — unsafe food may silently linger	State Diagram: * → Expired; Module §11.4	TC-15
FR-12	Usability — donor control over own pending items	Route: /donation/<id>/delete	Manual checklist

8. Data Flow Diagrams (DFD)

8.1 Level 0 — Context Diagram

The context diagram treats ChariSmart as a single process (0.0) and shows its data exchange with four external entities: Donor, Recipient, Admin and Volunteer/Rider.

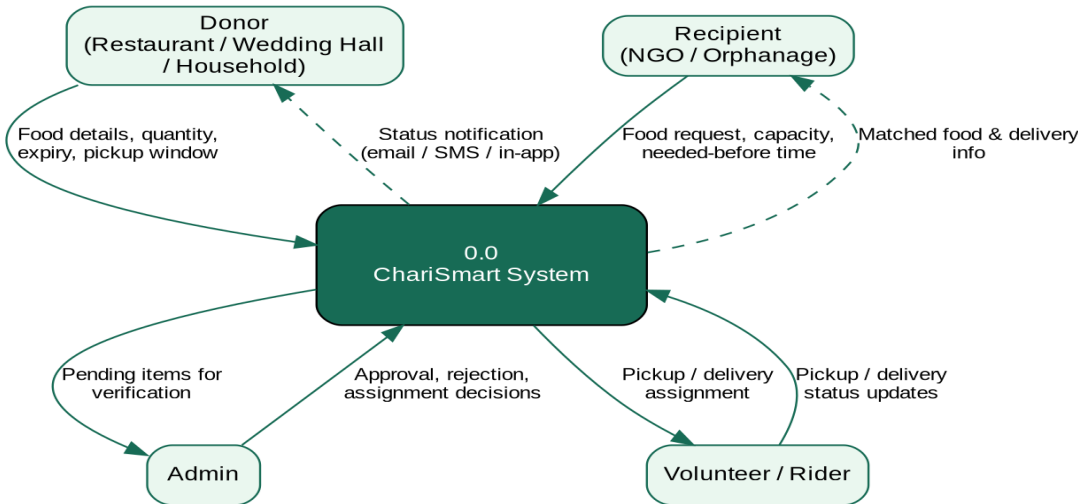


Figure 8.1 Level 0 context diagram of the ChariSmart system.

8.2 Level 1 — System Decomposition

Process 0.0 is decomposed into five sub-processes — 1.0 Manage Donation, 2.0 Manage Food Request, 3.0 Verify & Approve, 4.0 Match Food and 5.0 Pickup & Delivery Tracking — reading from and writing to five data stores (D1 Donations, D2 Food Requests, D3 Food Matches, D4 Notifications, D5 Users).

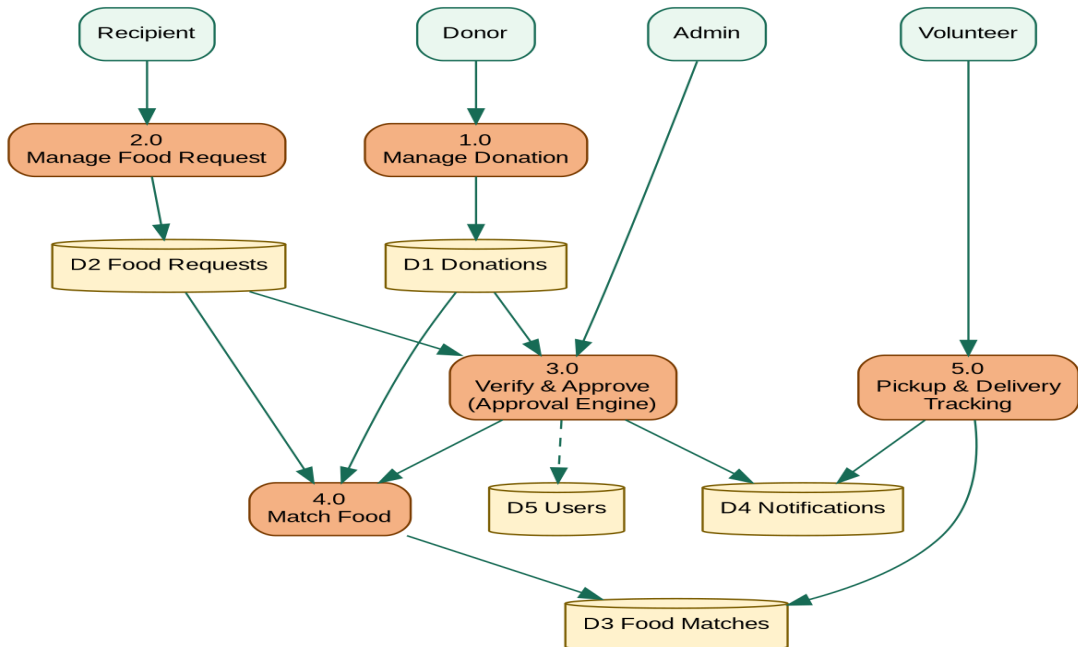


Figure 8.2 Level 1 decomposition of the ChariSmart system.

8.3 Level 2 — Approval & Notification Engine (P3 Detail)

Process 3.0 is further decomposed into five steps: 3.1 Validate Submission, 3.2 Record Admin Decision, 3.3 Update Status, 3.4 Generate Notification and 3.5 Dispatch Email/SMS. This corresponds directly to `decision_notification()` and `notify_user()` in `app.py`, which run immediately after every admin approve/reject action.

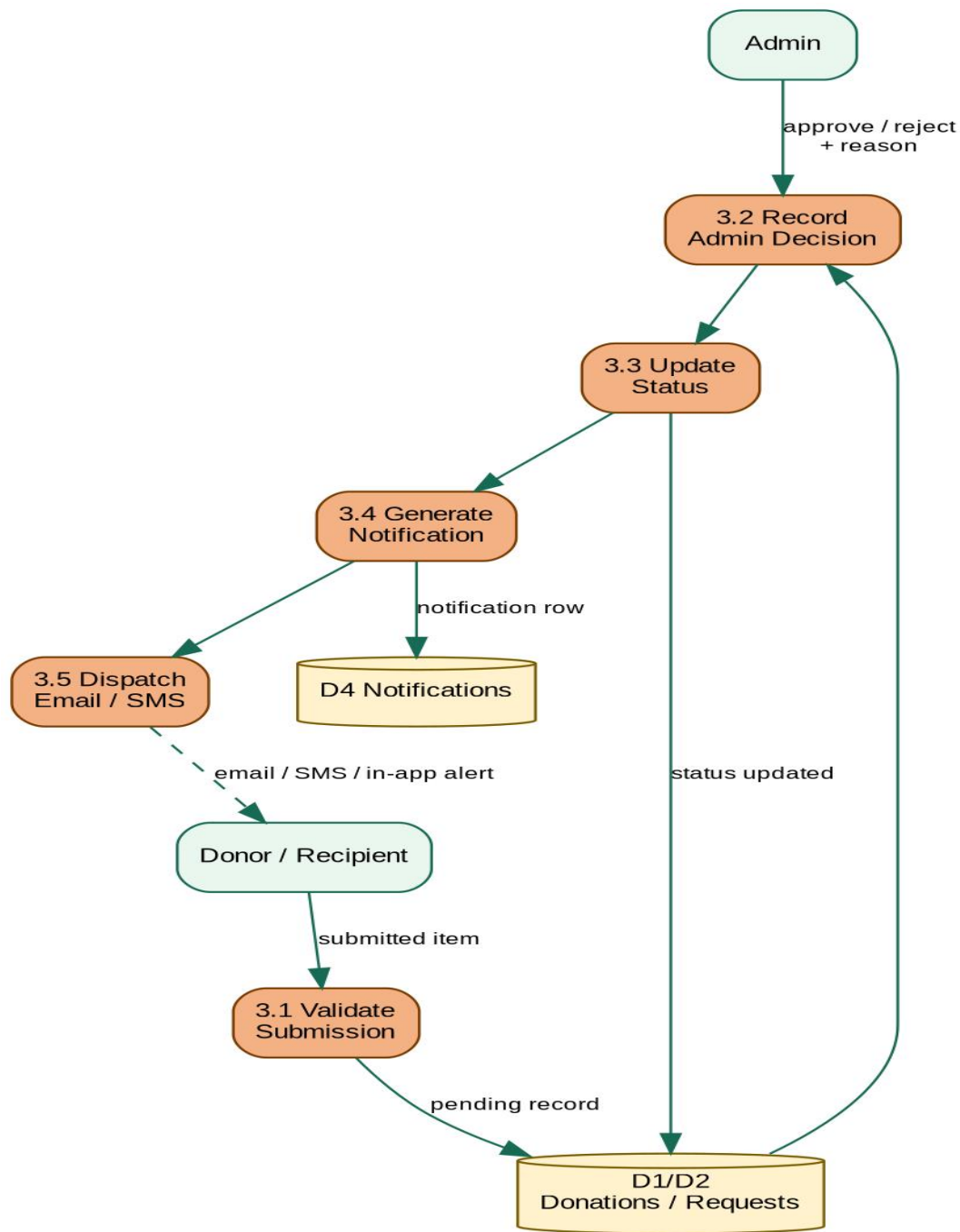
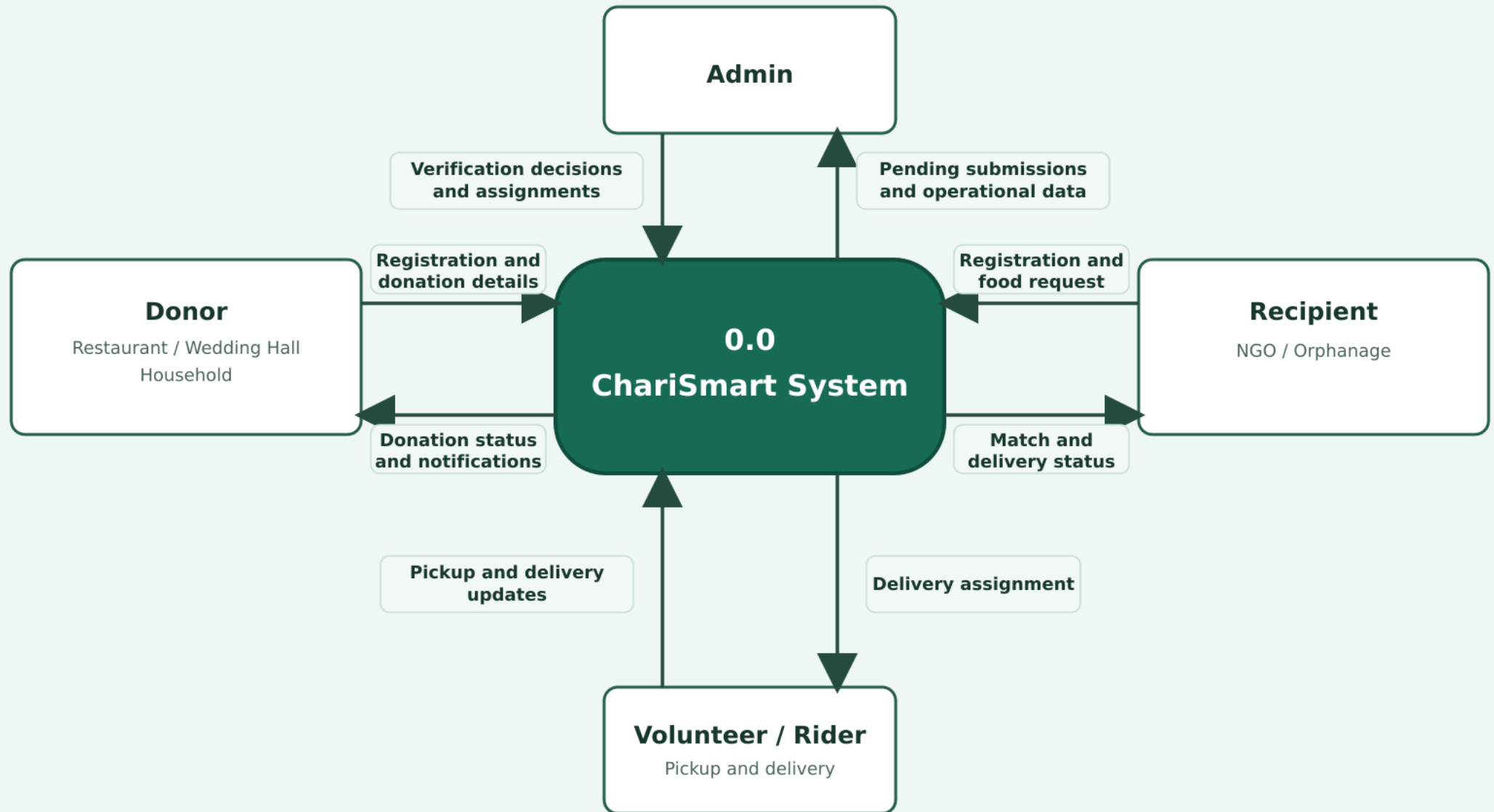


Figure 8.3 Level 2 decomposition of the Verify & Approve (P3.0) approval and notification engine.

ChariSmart Context DFD

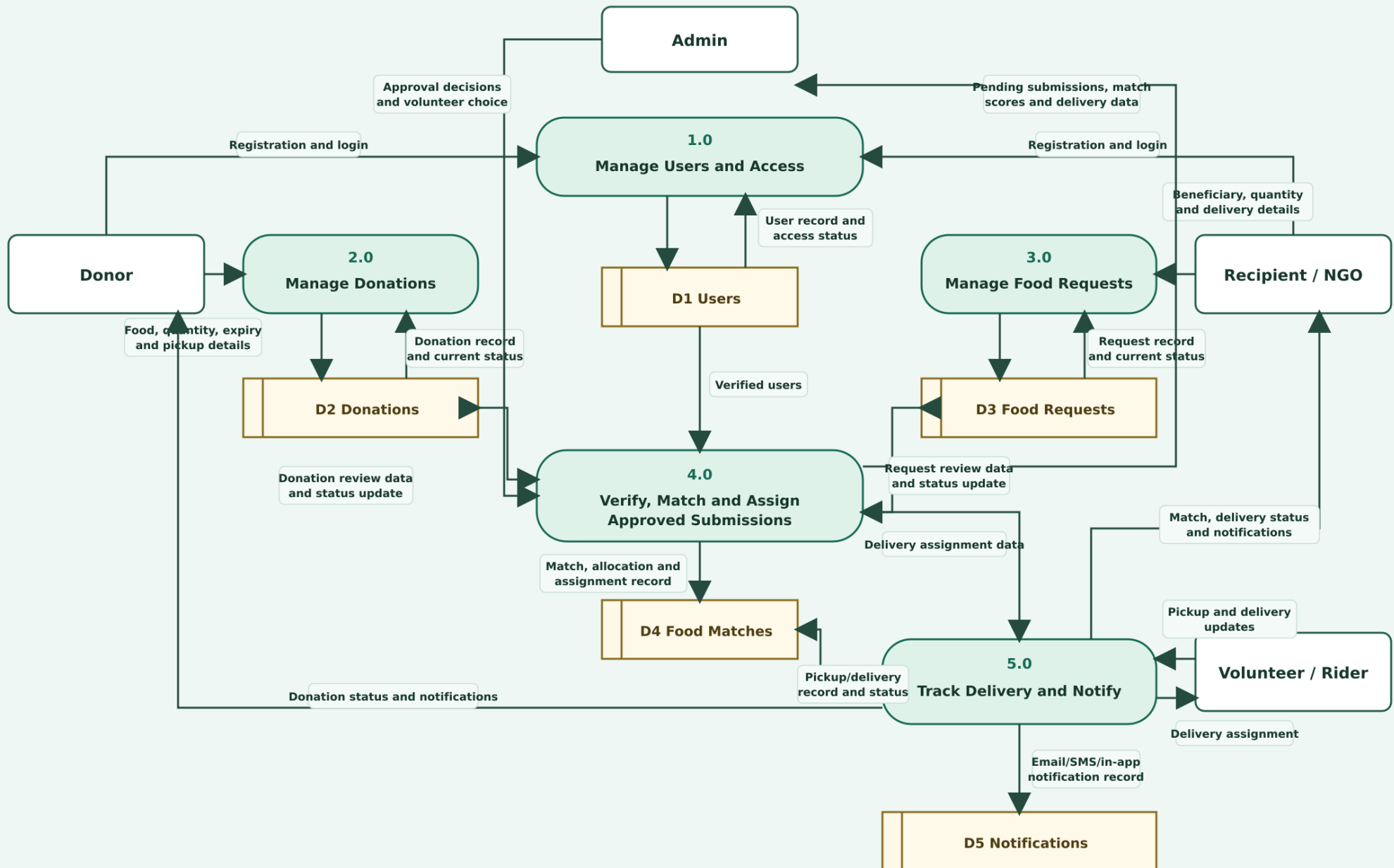
One central process with balanced inputs and outputs for all external entities



External entity = rectangle | Process = rounded green rectangle | Arrow = named data flow

ChariSmart Level-1 DFD

Balanced decomposition of access, donation, request, matching, delivery and notification functions



Rectangle = external entity | Rounded process = transformation | Open rectangle = data store | Named arrow = data flow

9. Entity-Relationship Diagram (ERD)

9.1 Entities, Attributes, and Keys

The section 6.2 ERD (Figure 6.2) is reproduced here for reference alongside a consolidated key listing:

Entity	Primary Key	Foreign Keys
USER	id	—
DONATION	id	donor_id → USER.id
FOOD_REQUEST	id	recipient_id → USER.id
FOOD_MATCH	id	donation_id → DONATION.id, request_id → FOOD_REQUEST.id, volunteer_id → USER.id (nullable)
NOTIFICATION	id	— (receiver_email logically references USER.email)

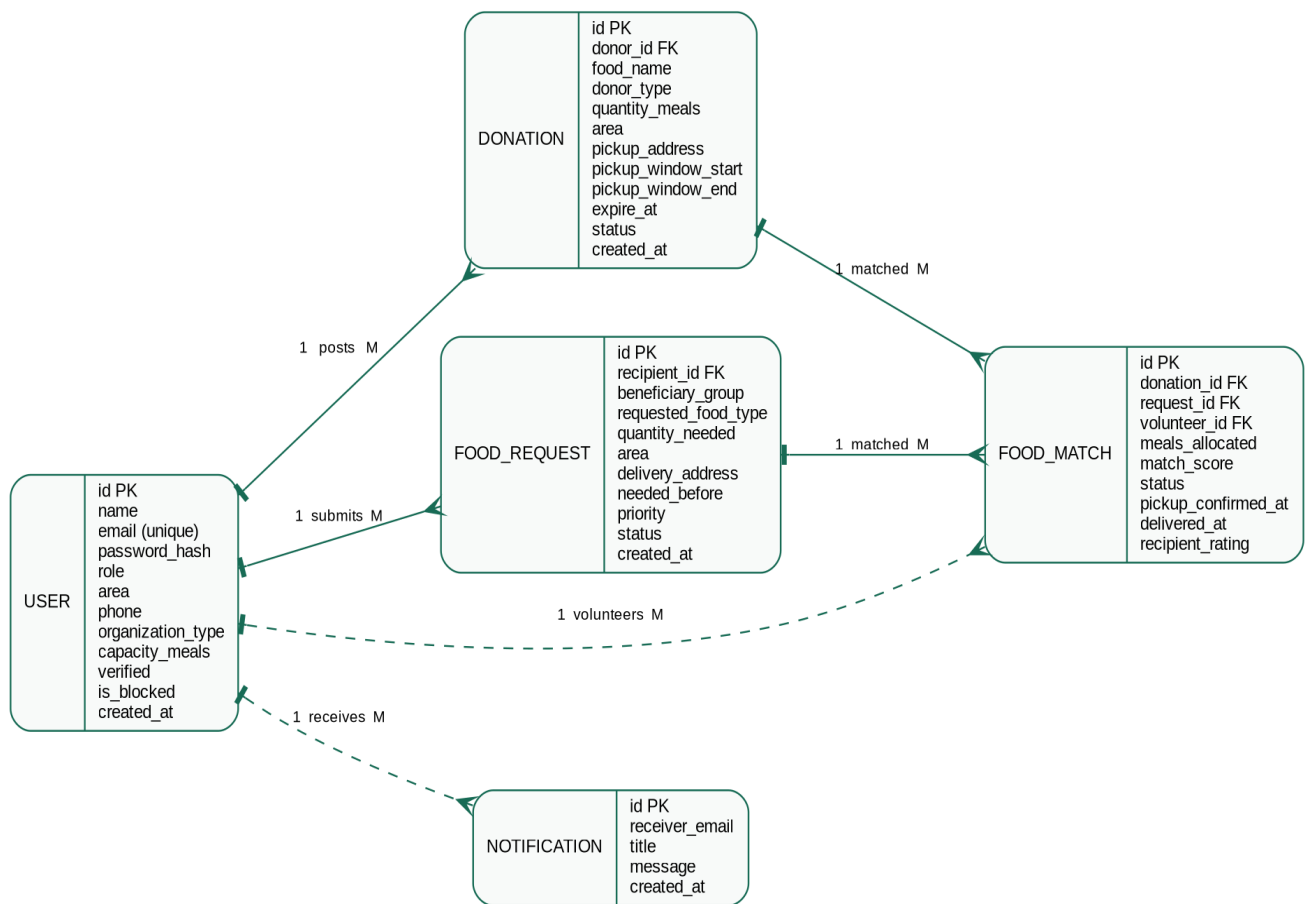


Figure 9.1 Entity-Relationship Diagram of the ChariSmart schema (repeated for reference).

10. UML Diagrams

10.1 Use Case Diagram — Summary

Four human actors — Donor, Recipient (NGO/Orphanage), Volunteer and Admin — plus one automated actor (the system scheduler that runs auto-expiry) interact with thirteen use cases inside the ChariSmart system boundary, including two <<include>> relationships (approval actions always include sending a notification) and one <<extend>> relationship (auto-matching optionally extends into volunteer assignment).

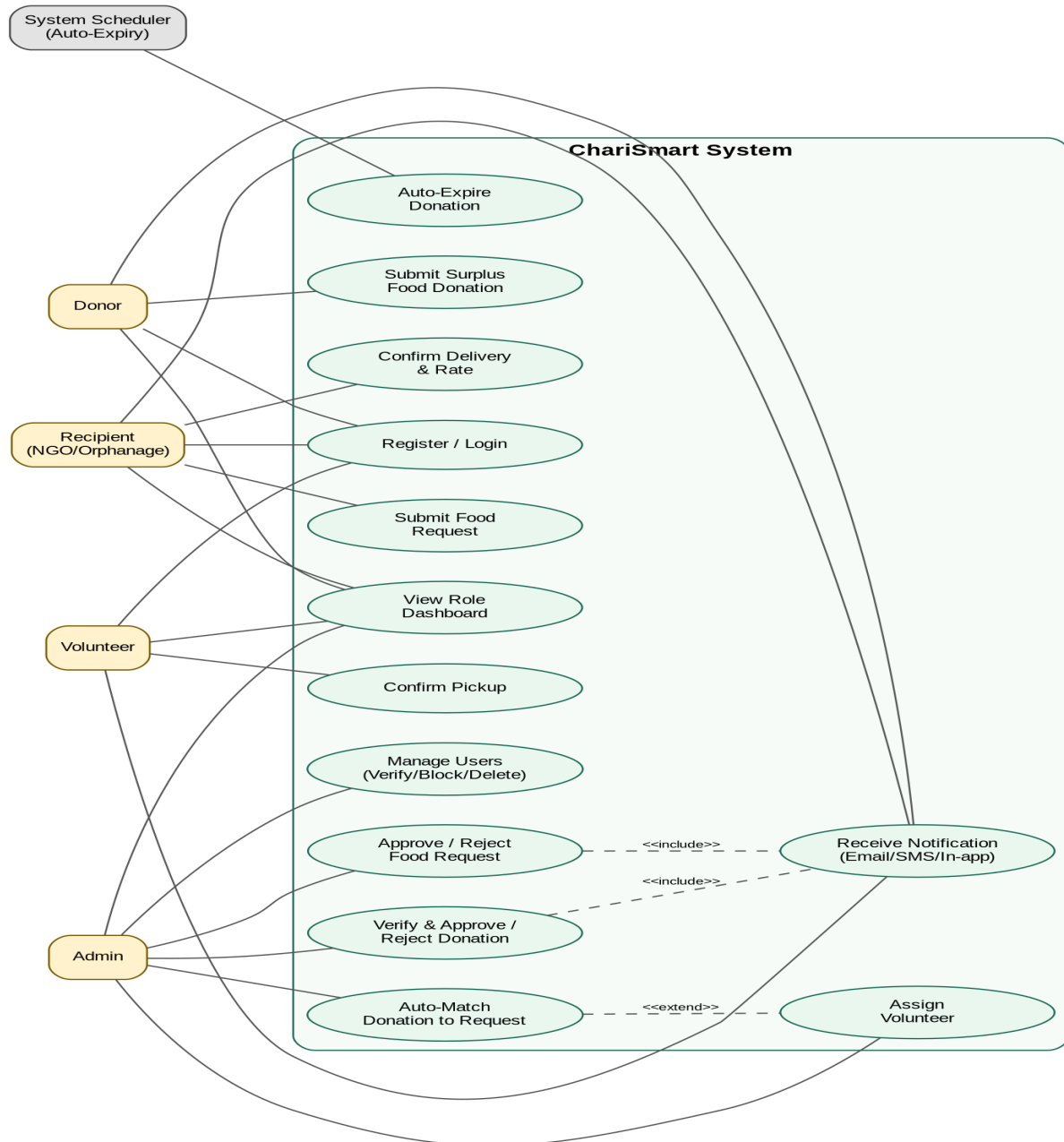


Figure 10.1 Use Case diagram — actors and use cases of the ChariSmart system.

10.2 Detailed Use Case Descriptions

Use Case: Submit Surplus Food Donation

Field	Description
Actor	Donor
Precondition	Donor is registered and logged in.
Main Flow	1) Donor opens the Donate form. 2) Enters food name, donor type, quantity, area, pickup address, pickup window and expiry. 3) Submits the form. 4) System validates the pickup window/expiry and stores the donation with status Pending.
Alternate Flow	If the pickup window or expiry is invalid (e.g. end before start, or expiry in the past), the system rejects the submission and shows a validation message.
Postcondition	A new Donation row exists with status Pending, visible to Admin for review.

Use Case: Verify & Approve / Reject Donation

Field	Description
Actor	Admin
Precondition	At least one Donation exists with status Pending.
Main Flow	1) Admin opens the Admin Dashboard. 2) Reviews a pending donation's details. 3) Clicks Approve (optionally with a note) or Reject (optionally with a reason).
Alternate Flow	Reject path records the reason and skips matching entirely.
Postcondition	Donation status becomes Approved or Rejected; a Notification is generated and dispatched to the donor (<<include>> Receive Notification).

Use Case: Auto-Match Donation to Request

Field	Description
Actor	Admin
Precondition	Donation is Approved; at least one FoodRequest is Approved and in the same normalised area.
Main Flow	1) Admin triggers auto-match for a donation. 2) System scores every candidate request (area match, quantity fit, priority) via calculate_match_score(). 3) System creates a FoodMatch with the best candidate and sets both donation and request status to Matched.
Alternate Flow	If no candidate request exists in the same area, the donation remains Approved and awaits a future request (<<extend>> — no match is forced).
Postcondition	A FoodMatch row exists with status Pending/Accepted, ready for volunteer assignment.
Postcondition	Match status reflects the real-world handover; the recipient can now confirm delivery and rate it.

10.3 Activity Diagram — Complete Donation Lifecycle

The activity diagram traces one donation from submission through the approve/reject decision, the area-match check, volunteer assignment, and finally the race between successful delivery and expiry.

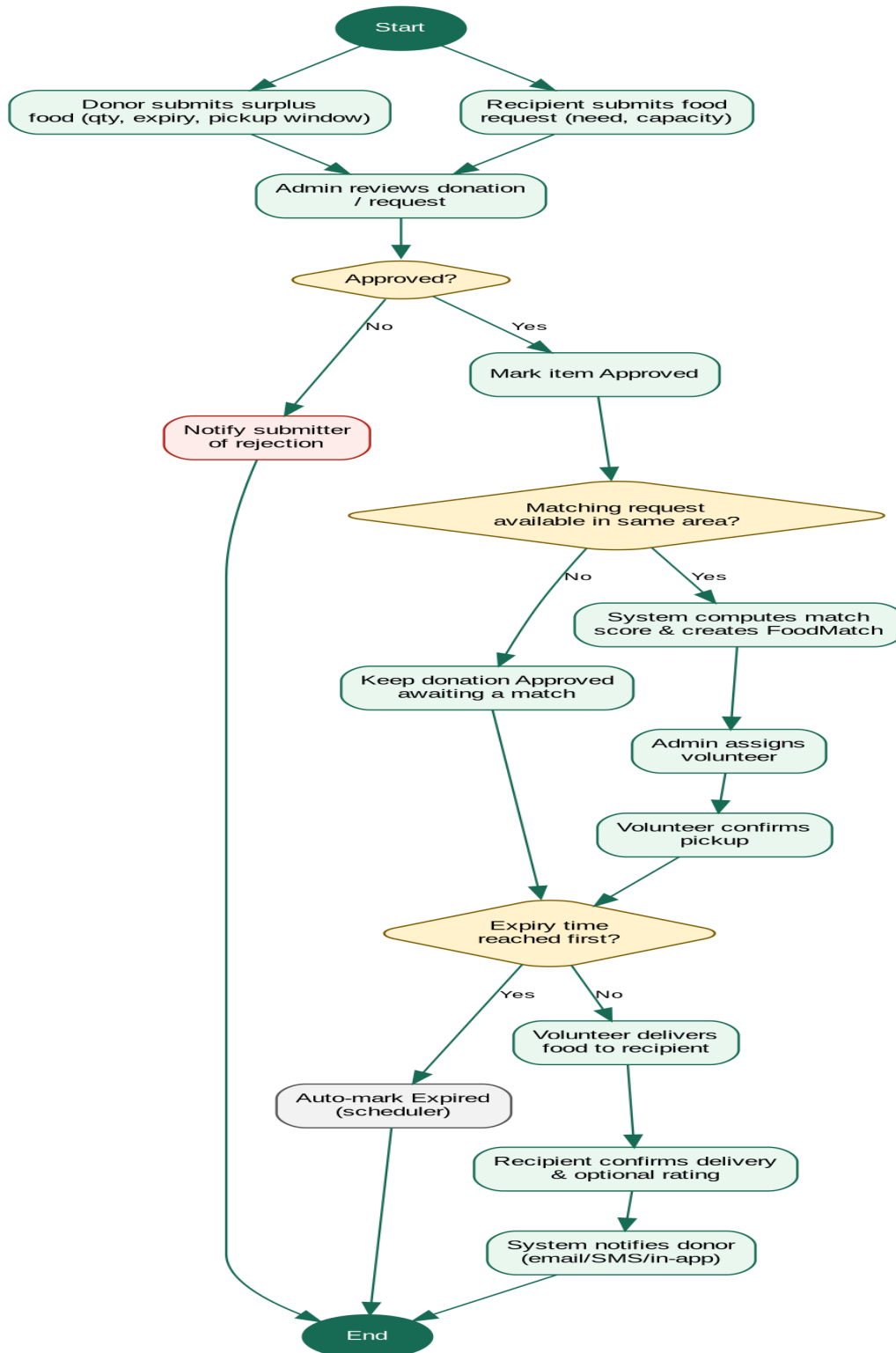


Figure 10.3 Activity diagram of the complete donation lifecycle.

10.4 Sequence Diagram — Admin Approves Donation

The sequence diagram follows one concrete scenario — an admin approving a donation — through validation, area-based matching, volunteer assignment, pickup, delivery and the resulting notification trail back to the donor and recipient.

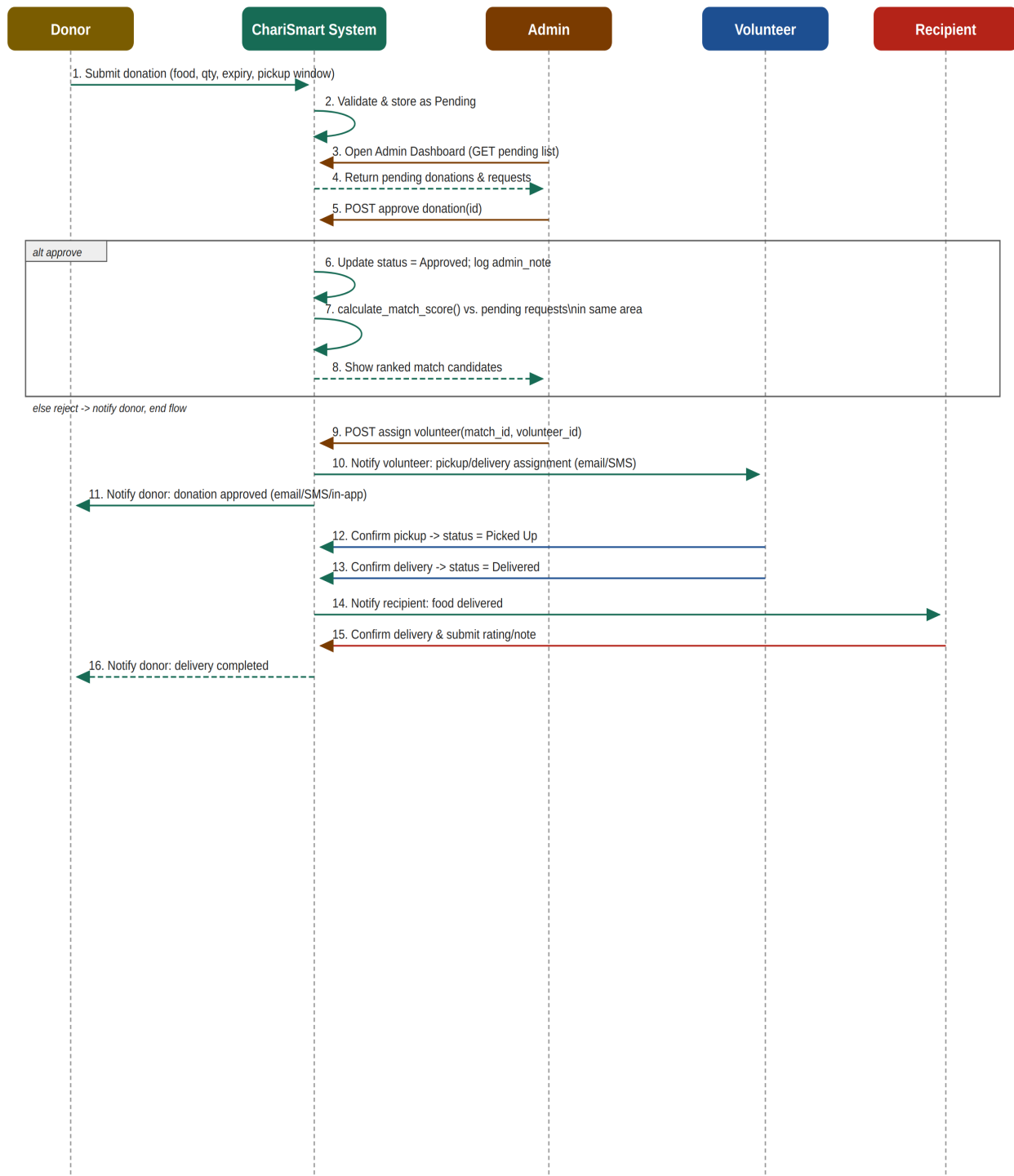


Figure: Sequence diagram for the "Admin Approves Donation" scenario, extended through matching, volunteer assignment and delivery confirmation.

Figure 10.4 Sequence diagram for the "Admin Approves Donation" scenario.

10.5 Class Diagram — Key Classes

The class diagram mirrors the five SQLAlchemy models plus a conceptual NotificationService that groups the email/SMS dispatch helpers (notify_user, decision_notification, send_email, send_sms) that live at module level in app.py.

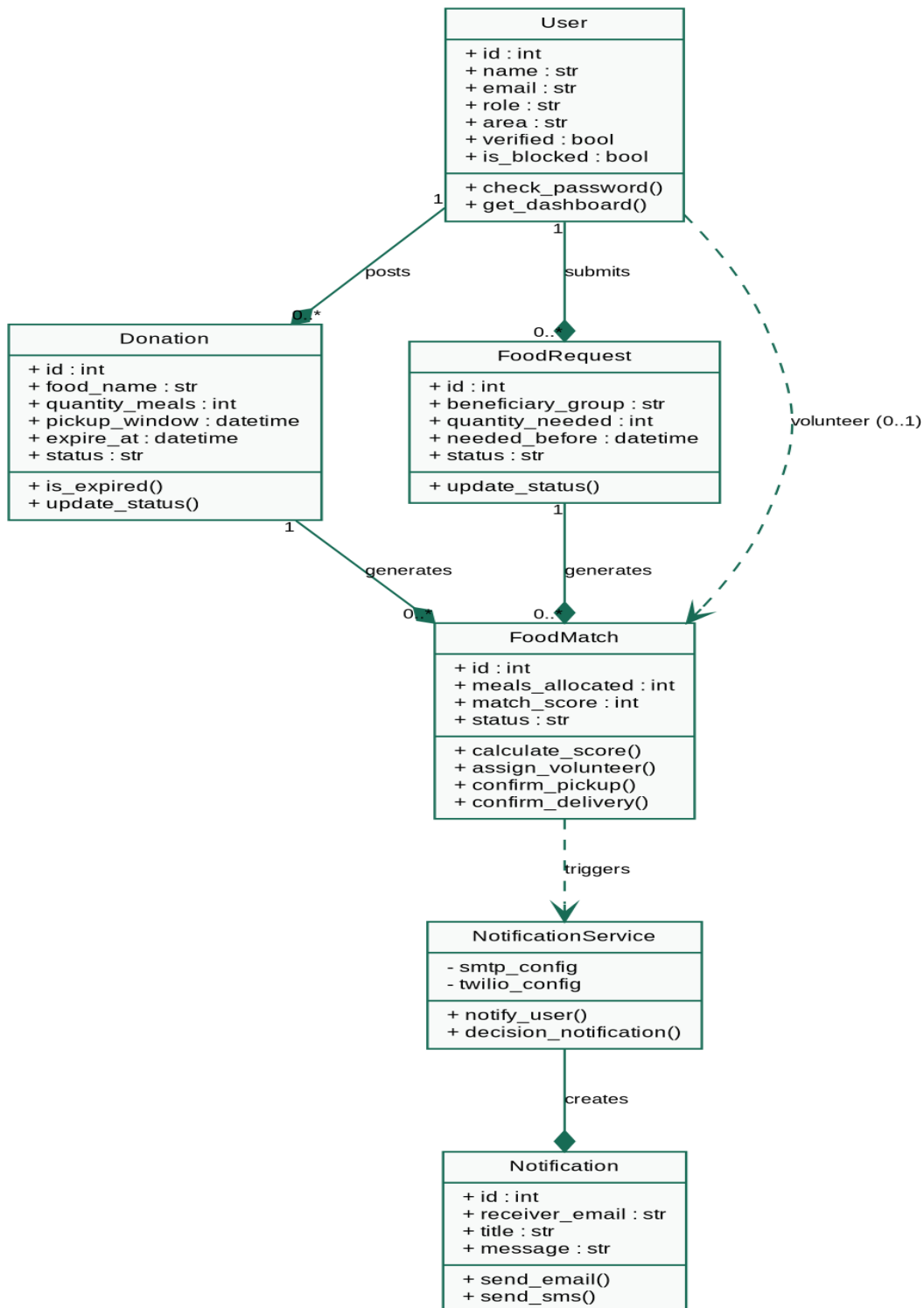


Figure 10.5 Class diagram of the key ChariSmart classes.

10.6 State Diagrams

A Donation (and, in parallel, the FoodMatch created from it) moves through a single well-defined state machine from submission to a terminal state:

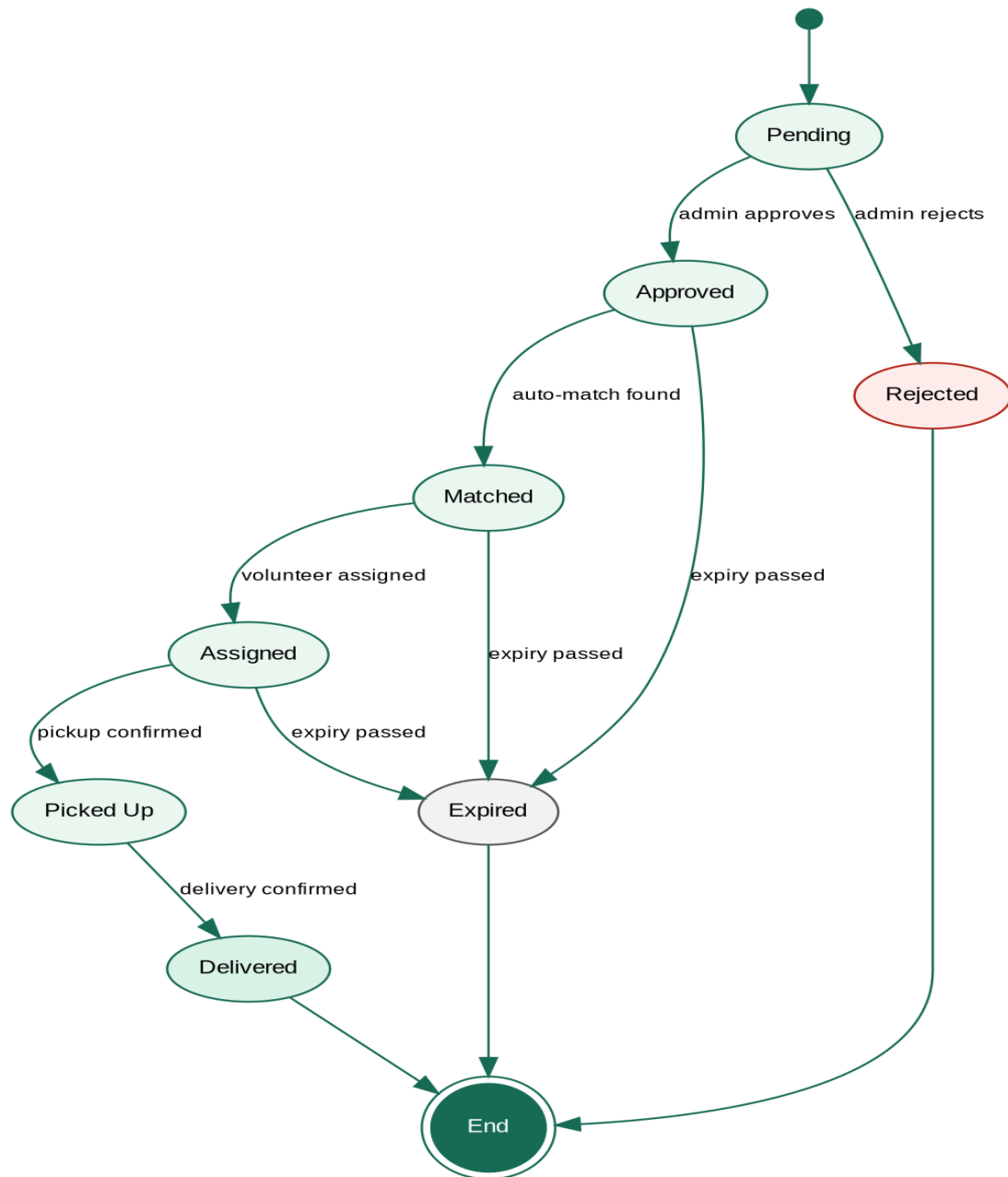


Figure 10.6 State diagram for the Donation / FoodMatch lifecycle.

10.7 Package Diagram

Although the prototype is implemented as a single app.py file for simplicity, it is internally organised into five logical layers/packages, shown below with their dependencies:

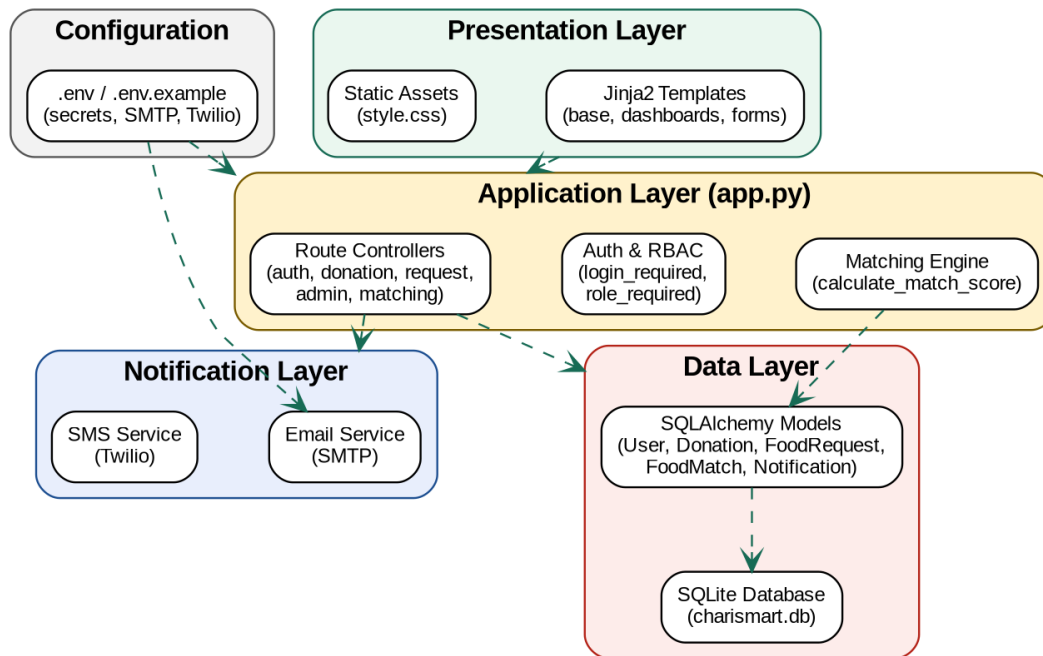


Figure 10.7 Package diagram showing the logical layering of the ChariSmart codebase.

11. Module & Feature Description

11.1 Authentication Module

- Registration (/register): creates a User with a hashed password, selected role, area, phone and (for donor/recipient) organization_type and capacity_meals.
- Login/Logout (/login, /logout): verifies credentials with check_password_hash and stores the user id in the Flask session; login_required protects every authenticated route.
- Role-based access control: role_required(role) wraps admin-only routes (approve/reject, assign volunteer, manage users) so non-admin users are redirected with a flash message.

11.2 Donation Module

- Donor submits a donation via /donate with quantity, area, pickup address, pickup window and expiry; parse_datetime()/parse_int() sanitise the form input.
- Admin reviews, approves or rejects via /admin/donation/<id>/<status>, optionally recording an admin_note or verification_note.
- Donor may delete their own still-pending donation via /donation/<id>/delete.

11.3 Notification & Email Module

- notify_user()/decision_notification() create a Notification row and call send_email() (Gmail SMTP via smtplib) and, if enabled, send_sms() (Twilio REST API).
- normalize_phone_for_sms() converts a Bangladeshi local number (017XXXXXXXX) to the international +88017XXXXXXXX format required by Twilio.
- When NOTIFICATION_EMAIL_ENABLED/NOTIFICATION_SMS_ENABLED are false or credentials are missing, the message is printed to the terminal instead of failing the request — keeping the demo workflow visible without exposing real credentials.

11.4 Auto-Expiry Module

- auto_expire_donations() runs on every request (via before_every_request) and marks any Approved/Matched/Assigned donation whose expire_at has passed as Expired, protecting recipients from unsafe, overdue food.

11.5 Food Request Module

- Recipient submits a request via /request-food with beneficiary_group, requested_food_type, quantity_needed, area, delivery_address, needed_before and priority.
- /requests lists requests for browsing; admin approves/rejects via /admin/request/<id>/<status>.
- calculate_match_score() and best_request_candidates() rank pending, approved requests in the same area against an approved donation for auto-matching.

12. UI/UX Design Description

12.1 Design Overview

The interface uses a calm, food-and-charity-appropriate palette — a deep green primary colour (#176B55) on a warm cream background (#F6F3EA) — with rounded cards, pill-shaped status badges and a sticky top navbar. Typography is a plain, highly legible Arial/Helvetica sans-serif stack for accessibility across low-bandwidth and older devices.

12.2 Template Structure

base.html defines the shared navbar, flash-message area and footer that every other template extends: home.html (public landing/hero), login.html and register.html (auth forms with a two-column auth-info/form-card layout), donate.html and request_food.html (submission forms with a responsive grid-form), donor_dashboard.html, recipient_dashboard.html, volunteer_dashboard.html and admin_dashboard.html (role-specific views), requests.html (browsing list) and manage_users.html (admin user table).

12.3 Pages and Key UI Elements

Page	Key Elements
Home	Hero section with headline and call-to-action buttons; an impact-card highlighting platform stats.
Register / Login	Split layout: branded info panel + form card; role selector; validation flash messages.
Donate / Request Food	Two-column grid-form for structured data entry (quantity, area, address, date/time windows).
Donor / Recipient / Volunteer Dashboard	stats-grid summary cards + cards-grid or table-card listing the user's own donations/requests/matches with status badges.
Admin Dashboard	Consolidated stats-grid plus table-cards for pending donations, pending requests and active matches, each with inline approve/reject/assign action buttons.
Manage Users	Table-card listing all users with verify/block/delete action buttons per row.

12.4 Status Badge Colours

Status Group	Colour Treatment
approved, accepted, delivered, fulfilled	Green badge (background #DCFCE7, text #166534) — positive/complete states.
pending, matched, assigned, picked_up	Blue badge (background #DBEAFE, text #1D4ED8) — in-progress states.
rejected, expired, cancelled	Red badge (background #FEE2E2, text #991B1B) — negative/terminal states.

12.5 User Personas

Persona	Goals	Pain Points Addressed
Rahim — Restaurant Manager (Donor)	Quickly log leftover catering food before it spoils.	No more phone-tag with NGOs; one form captures everything a volunteer needs.
Nasrin — NGO Coordinator (Recipient)	See genuinely available, verified food nearby before it expires.	No wasted trips chasing donations that are already claimed or unsafe.
Karim — Volunteer/Rider	Get a clear, single assignment with pickup and delivery details.	No ambiguity about what to collect, from where, and by when.
Admin (Course/Ops Staff)	Keep the whole pipeline safe, verified and moving.	One dashboard to verify, approve, match and assign, instead of scattered messages.

12.6 Simplified Navigation Flow

- 13.Home → Register/Login → role-specific Dashboard.
- 14.Donor Dashboard → Donate form → back to Dashboard (new item shows as Pending).
- 15.Recipient Dashboard → Request Food form → Requests list → Dashboard (status updates as Admin acts).
- 16.Admin Dashboard → review pending donation/request → Approve/Reject → (if approved) Auto-Match → Assign Volunteer.
- 17.Volunteer Dashboard → assigned match → Confirm Pickup → Confirm Delivery.
- 18.Recipient Dashboard → Confirm Delivery & Rate → cycle complete.

12.7 Accessibility Notes

- High colour contrast between text and background (dark ink #1F2933 on cream/white cards) supports readability.
- Status is conveyed by both colour and text label (e.g. the word 'Approved' plus a green badge), not colour alone, for colour-blind users.
- A responsive @media(max-width:760px) breakpoint collapses multi-column grids (hero, auth layout, forms) to a single column for mobile use in the field by volunteers.

13. Implementation Details

13.1 Running the Application

From the project root (ChariSmart_notifications/):

- Create and activate a virtual environment: `python -m venv venv`, then `venv\Scripts\activate` (Windows) or `source venv/bin/activate` (macOS/Linux).
- Install dependencies: `pip install -r requirements.txt`.
- Copy environment template: `copy .env.example .env` (Windows) or `cp .env.example .env`.
- Run the app: `python app.py`, then open `http://127.0.0.1:5000`.
- Default admin login: `admin@example.com` / `admin123` (seeded automatically).
- Set `SEED_DEMO=true` before first run to also seed demo donor/recipient/volunteer accounts.

13.2 Environment Variables

Variable	Purpose
SECRET_KEY	Flask session signing key.
DATABASE_URL	SQLAlchemy connection string (defaults to local SQLite file).
NOTIFICATION_EMAIL_ENABLED	Turns on real SMTP email sending.
MAIL_SERVER / MAIL_PORT / MAIL_USERNAME / MAIL_PASSWORD / MAIL_FROM	Gmail SMTP configuration (use an app password, not the normal Gmail password).
EMAIL_USE_TLS / EMAIL_USE_SSL	Transport security flags for the SMTP connection.
NOTIFICATION_SMS_ENABLED / SMS_PROVIDER	Turns on SMS sending and selects the provider (twilio).
TWILIO_ACCOUNT_SID / TWILIO_AUTH_TOKEN / TWILIO_FROM_NUMBER	Twilio credentials for SMS dispatch.
SEED_DEMO	When true, seeds demo donor/recipient/volunteer accounts alongside the default admin.

13.3 Key Implementation Patterns

- Decorator-based cross-cutting concerns: `@login_required` and `@role_required(role)` wrap view functions to enforce authentication and authorisation without repeating checks in every route.
- Graceful degradation for notifications: `send_email()/send_sms()` catch connection/credential errors and fall back to printing the message to the terminal, so the demo never crashes on missing secrets.
- Single source of truth for vocabularies: `ROLES`, `DONOR_TYPES`, `DONATION_STATUSES`, `REQUEST_STATUSES` and `MATCH_STATUSES` are defined once as module-level lists and reused everywhere a value is validated or rendered.

- Continuous housekeeping: `before_every_request()` calls `auto_expire_donations()` on every request instead of requiring a separate cron job/scheduler process.
- Area-normalised matching: `normalize_area()` is applied consistently before storage and before comparison so the auto-match query is a reliable equality check rather than a fuzzy one.

13.4 Coding Standards

- PEP 8-style naming: `snake_case` for functions/variables, `PascalCase` for SQLAlchemy model classes.
- Environment-driven configuration: no secret ever hard-coded in `app.py`; all credentials come from `os.getenv()` with safe defaults for local development.
- Defensive parsing: `parse_datetime()/parse_int()` centralise form-input parsing and fall back safely instead of raising unhandled exceptions on malformed input.

13.5 Folder Structure

- `app.py` — single-file Flask application (models, helpers, routes).
- `templates/` — Jinja2 HTML templates (base, home, login, register, donate, request_food, requests, donor/recipient/volunteer/admin dashboards, manage_users).
- `static/style.css` — the entire visual design system (CSS variables, components, responsive rules).
- `tests/` — `test_notification_flow.py` (automated) and `test_manual_checklist.md` (manual QA checklist).
- `docs/` — this final report, the SRS, diagram sources (`docs/diagrams/*.mmd`, `docs/diagram_images/*`), and the defense presentation.
- `.env.example` / `.gitignore` — configuration template and version-control hygiene (the real `.env` is never committed).

14. Testing Strategy

14.1 Testing Approach

Testing combined an automated script (tests/test_notification_flow.py) that exercises the notification pathway in isolation, a Python syntax check (py_compile) across the codebase, and a structured manual test checklist (tests/test_manual_checklist.md) covering every role and every status transition end to end through the running application.

14.2 Test Cases

TC ID	Scenario	Expected Result	Status
TC-01	Admin login	Admin dashboard opens	Pass
TC-02	Donor registration	Donor account created	Pass
TC-03	Recipient registration	Recipient account created	Pass
TC-04	Volunteer registration	Volunteer account created	Pass
TC-05	Admin verification toggle	User verification status changes	Pass
TC-06	Donation submission	Donation created with status Pending	Pass
TC-07	Invalid pickup window	Submission blocked with validation message	Pass
TC-08	Request submission	Request created with status Pending	Pass
TC-09	Approve request	Request status becomes Approved	Pass
TC-10	Approve donation	Donation status becomes Approved	Pass
TC-11	Auto-match	FoodMatch created between compatible donation and request	Pass
TC-12	Assign volunteer	Match status becomes Assigned	Pass
TC-13	Pickup update	Match status becomes Picked Up	Pass
TC-14	Delivery confirmation	Donation marked Delivered and request marked Fulfilled	Pass
TC-15	Auto-expiry	Overdue donation automatically marked Expired	Pass

14.3 Additional Testing Notes

- Syntax verification completed with Python's py_compile across app.py before every functional test pass.
- The full manual checklist is maintained in tests/test_manual_checklist.md and should be re-run after any schema or route change.
- For the final submission, the application should be run locally end-to-end to capture live dashboard screenshots alongside this report.

15. Risk Analysis & Mitigation

Risk	Likelihood	Impact	Mitigation
Donor enters false/inflated food details	Medium	High (food safety)	Admin verification step required before any donation can be matched; verified-donor flag on User.
Pickup missed before expiry	Medium	High (wasted/unsafe food)	Explicit pickup_window fields plus an automatic Expired transition removes the item from circulation.
Volunteer unavailable at assignment time	Medium	Medium	Admin manually selects from available volunteers rather than relying on a rigid auto-dispatch.
SMTP/Twilio not configured during a demo or in production	High (demo), Low (prod)	Low	Notification functions fall back to a terminal-printed message instead of failing the request.
Area free-text mismatch breaks matching	Low	Medium	normalize_area() trims/lower-cases area on write and on compare to keep matching reliable.
Single SQLite file as a scaling/concurrency bottleneck	Low (prototype), Medium (scale-up)	Medium	DATABASE_URL abstraction allows a drop-in move to PostgreSQL/MySQL without code changes.
Secrets committed to version control	Low	High	.env is git-ignored; only .env.example (placeholders) is tracked.
No independent food-safety certification	Medium	High	Explicitly documented as a legal/ethical limitation (§4.5); admin verification is a human control, not a certification substitute.

16. References

1. Sommerville, I. Software Engineering. Pearson.
2. Pressman, R. S. Software Engineering: A Practitioner's Approach. McGraw-Hill.
3. Dennis, A., Wixom, B. H., and Roth, R. M. Systems Analysis and Design. Wiley.
4. Flask Documentation. Pallets Projects. <https://flask.palletsprojects.com>
5. Flask-SQLAlchemy Documentation. Pallets Projects. <https://flask-sqlalchemy.palletsprojects.com>
6. SQLAlchemy Documentation. SQLAlchemy Project. <https://www.sqlalchemy.org>
7. Werkzeug Security Documentation. Pallets Projects.
8. Twilio Programmable SMS API Documentation. <https://www.twilio.com/docs/sms>
9. Python smtplib Documentation. <https://docs.python.org/3/library/smtplib.html>
10. ChariSmart project source code, README.md and docs/ folder (this submission).